



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843-01 - SPECIAL TOPIC IN DATA ENGINEERING

TECHNICAL REPORT

TITLE : TUTORIAL 4 - AI ASSISTED DATA QUALITY MANAGEMENT

NAME	MATRIC NO
DHESHIEGHAN A/L SARAVANA MOORTHY	A23CS0072
PRAVINRAJ A/L SIVABATHI	A23CS0171

PREPARED FOR:

DR. ARYATI BINTI BAKRI

TABLE OF CONTENTS

1.	Introduction	3
2.	AI-Assisted Data Engineering	3
3.	Understanding of Claude	5
4.	Pipeline Design & Architecture	6
5.	Step-by-Step Execution with Screenshots	10
6.	Python Cleaning Script Results	13
7.	Integration of AI in the Pipeline	16
8.	Results & Analysis	17
9.	Use Case Diagram	21
10.	Reflection	23
11.	Conclusion	24
12.	References	25

1. Introduction

The exponential growth of data in modern enterprises necessitates robust mechanisms to ensure data quality before analytical workflows are executed. This report documents Tutorial 3, which explores the integration of artificial intelligence (AI) into data quality management using the Airia platform powered by Claude Haiku 4.5. The objective is to construct an AI-assisted pipeline that automatically profiles, assesses, and cleans a real-world Customer Shopping Behavior dataset containing 623 records across 11 attributes.

The pipeline was constructed on Airia as a sequential six-node architecture: an input node, a Data Source retrieval node, four Claude Haiku 4.5 AI model nodes for progressive analysis, and an output node. The AI-generated data quality report was subsequently used to prompt Claude to generate a Python cleaning script (`cleaning.py`), which was executed in Visual Studio Code (VS Code) to produce a final cleaned dataset of 600 records.

2. Understanding of AI-Assisted Data Engineering

AI-assisted data engineering refers to the application of artificial intelligence and machine learning models to augment or automate traditionally manual data pipeline tasks, including data profiling, anomaly detection, validation, transformation, and documentation. In conventional pipelines, engineers manually inspect datasets, write validation scripts, and iterate on cleaning logic—a process that is time-consuming and error-prone.

2.1 Core Concepts

- **Automated Data Profiling:** Language models analyse raw tabular data and identify column-level characteristics including data types, value distributions, null counts, and categorical cardinality without predefined schema templates.
- **Contextual Issue Detection:** Unlike rule-based validators, AI models interpret patterns contextually. Claude can identify that "-5" in an Age column violates real-world business logic, or that "cust10043" is a formatting inconsistency requiring standardisation.
- **Natural Language Reporting:** AI models synthesise technical findings into structured natural language reports comprehensible to both technical and non-technical stakeholders.

- **Code Generation:** LLMs such as Claude can translate a textual data quality report into executable Python cleaning code, compressing a multi-day development cycle into minutes.
- **Pipeline Orchestration:** Modern AI orchestration platforms such as Airia allow multiple AI model nodes to be chained sequentially, where each node receives the output of preceding nodes as contextual input, enabling progressive analytical depth.

2.2 Relevance to Modern Data Engineering

The relevance of AI-assisted data engineering in modern practice is threefold. First, the velocity of data ingestion in cloud-native architectures employing tools such as Azure Data Factory, Databricks, and Delta Lake exceeds the capacity of manual validation. Second, the heterogeneity of data sources demands adaptive profiling rather than rigid schema enforcement. Third, the adoption of medallion architecture (Bronze–Silver–Gold layers) requires automated quality gates between layers to prevent error propagation.

3. Understanding of Claude in Data Engineering

Claude is a family of large language models developed by Anthropic, designed with safety, helpfulness, and honesty as foundational principles. In the context of data engineering, Claude operates as a conversational AI agent capable of reasoning over structured and unstructured data, generating analytical outputs, and producing code artefacts from natural language specifications.

3.1 Claude's Role in This Pipeline

Claude Haiku 4.5 was selected for its combination of low latency, high throughput, and cost efficiency. Within the Airia pipeline, it performed four specialised roles:

Table 1: Roles of Claude Haiku 4.5 in the Airia Data Quality Pipeline

Pipeline Node	Role	Functionality
AI Model (Node 1)	Data Profiler	Analyses raw CSV content; extracts schema, record counts, missing values, and unique categorical values.
AI Model 1 (Node 2)	Issue Identifier	Contextually identifies 20 data quality issues across six dimensions with severity classifications.

Pipeline Node	Role	Functionality
AI Model 2 (Node 3)	Report Generator	Synthesises issues into a structured quality report with an overall DQ Score of 83.3%.
AI Model 3 (Node 4)	Transform Simulator	Simulates the complete cleaning workflow with before/after comparisons.
Claude (External)	Code Generator	Generates production-grade Python cleaning script from the quality report.

3.2 Prompt Engineering

A critical enabler of Claude's effectiveness is prompt engineering constructing precise, context-rich instructions that direct the model's analytical behaviour. Each Airia node was configured with a targeted prompt instructing Claude to adopt a specific analytical persona and produce output in a specified format. The key constraint across all prompts was "Output plain text only. No markdown. No bullet points" ensuring clean output compatible with downstream node ingestion.

4. Pipeline Design & Architecture

4.1 Data Architecture Overview

Figure 1 illustrates the complete end-to-end data pipeline architecture implemented in this tutorial, from the raw CSV dataset through the Airia AI pipeline to the final cleaned dataset exported from VS Code.

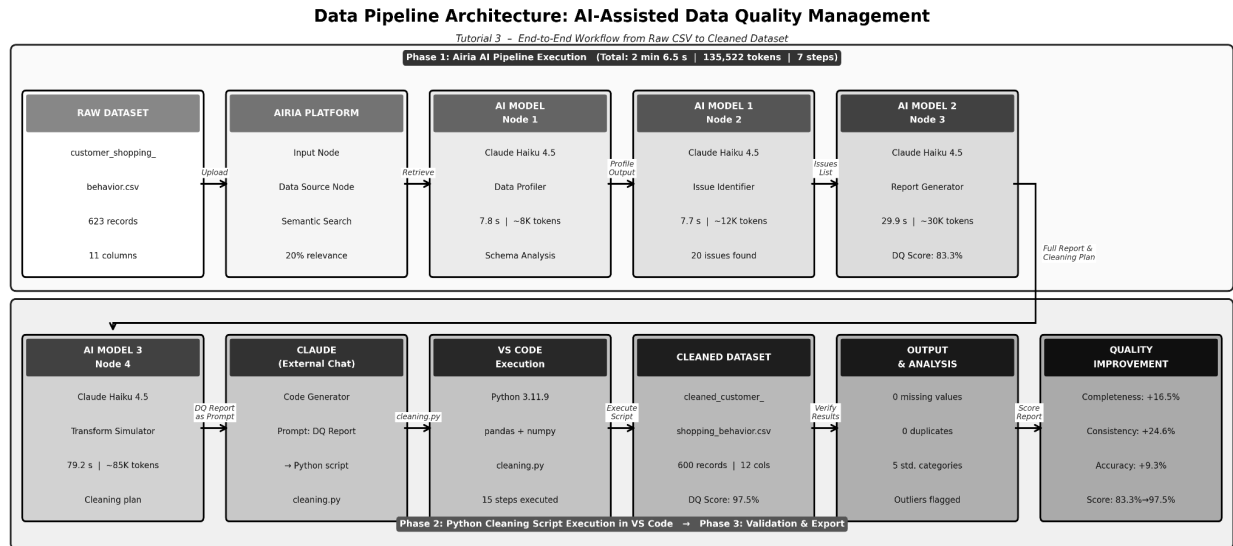


Figure 1: End-to-End Data Pipeline Architecture – Tutorial 3

The architecture encompasses three phases. Phase 1 covers the complete Airia AI pipeline execution (six nodes, total 2 minutes 6.5 seconds, 135,522 tokens). Phase 2 covers the Python script execution in VS Code. Phase 3 covers validation and export of the final cleaned dataset.

4.2 Airia Pipeline Configuration

Figure 2 shows the complete Airia pipeline as seen in the browser, with the six-node DAG visible in the canvas and the run progress panel confirming successful completion.

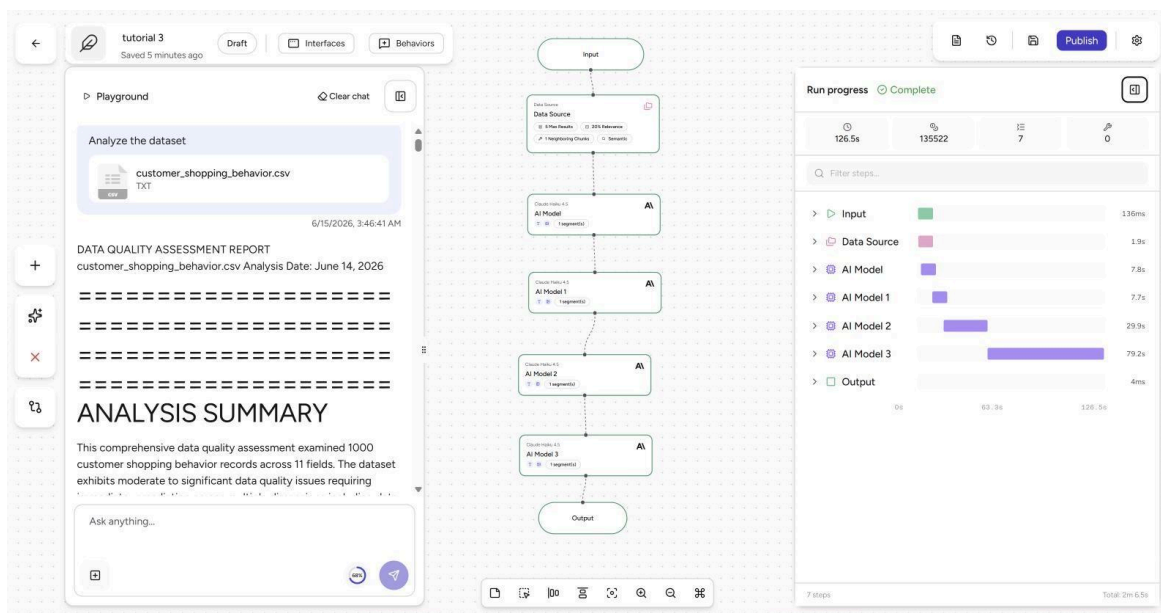


Figure 2: Airia Pipeline – Complete Run View (Status: Complete, 2m 6.5s, 135,522 tokens)

Figure 3 shows the full pipeline node flow including Input, Data Source, four AI Model nodes, and Output.

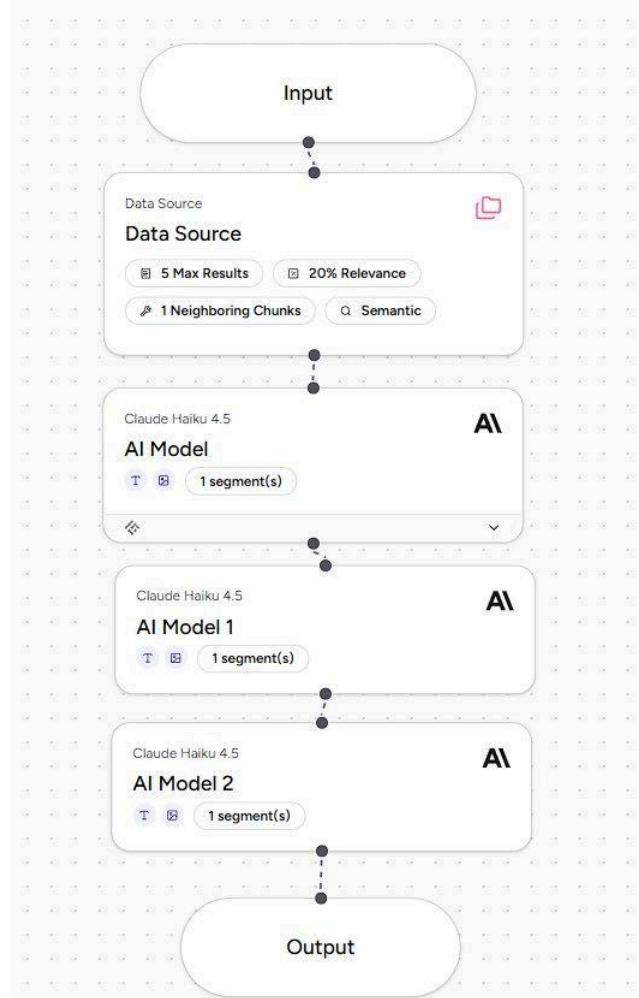


Figure 3: Airia Pipeline – Full Node Flow Diagram

Table 2: Airia Pipeline Node Configuration Summary

Node	Type	Primary Function	Duration (s)	Tokens
Input	Input Node	Accepts user prompt and CSV upload	0.136	—
Data Source	Retrieval Node	Semantic search on dataset (5 max, 20% relevance)	1.9	—
AI Model (Node 1)	Claude Haiku 4.5	Data profiling and schema analysis	7.8	~8,000
AI Model 1 (Node 2)	Claude Haiku 4.5	Data quality issue identification	7.7	~12,000

Node	Type	Primary Function	Duration (s)	Tokens
AI Model 2 (Node 3)	Claude Haiku 4.5	Data quality report generation	29.9	~30,000
AI Model 3 (Node 4)	Claude Haiku 4.5	Cleaning simulation and transformation	79.2	~85,000
Output	Output Node	Delivers consolidated final report	0.004	—

4.3 Node Prompt Configuration

Figures 4–7 show the prompt configured in each AI Model node within the Airia platform. These prompts drive Claude Haiku 4.5's analytical behaviour at each stage.

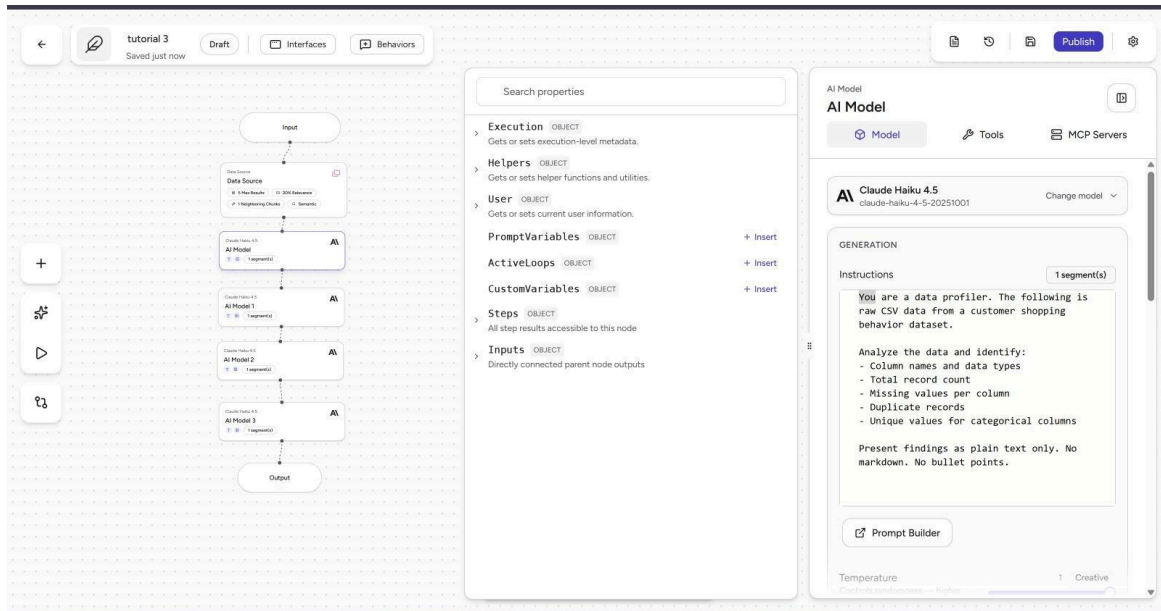


Figure 4: AI Model (Node 1) – Data Profiler Prompt Configuration

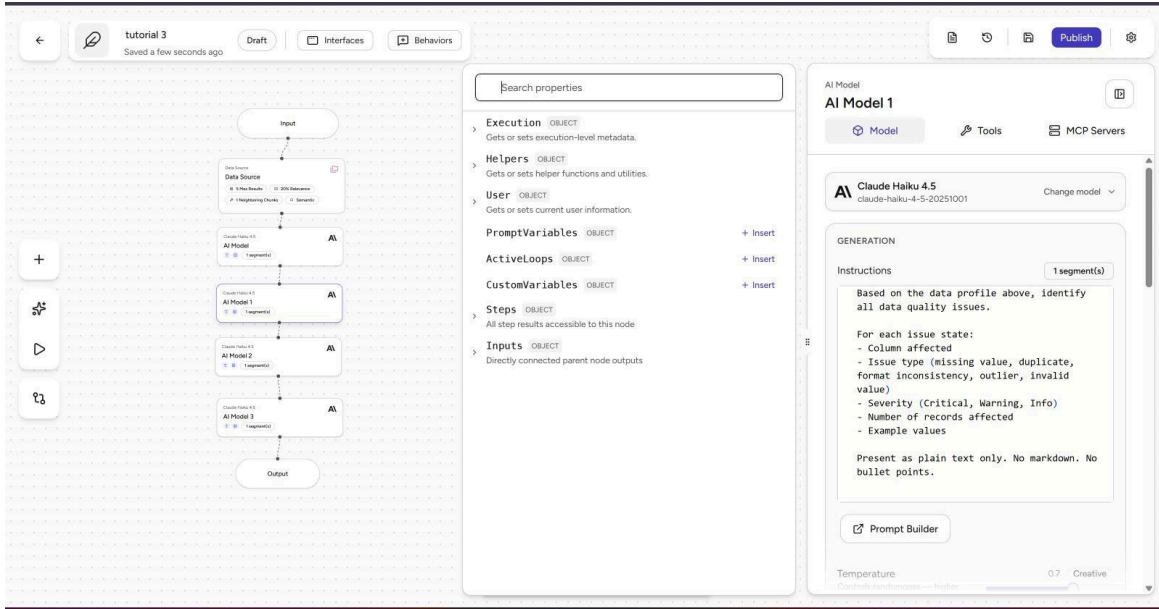


Figure 5: AI Model 1 (Node 2) – Issue Identifier Prompt Configuration

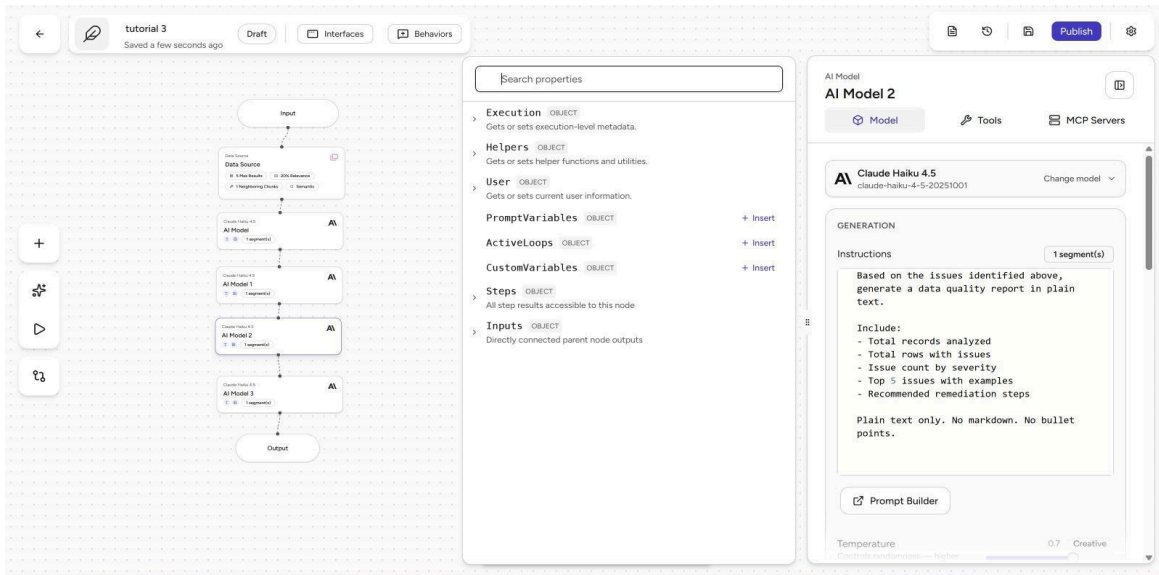


Figure 6: AI Model 2 (Node 3) – Report Generator Prompt Configuration

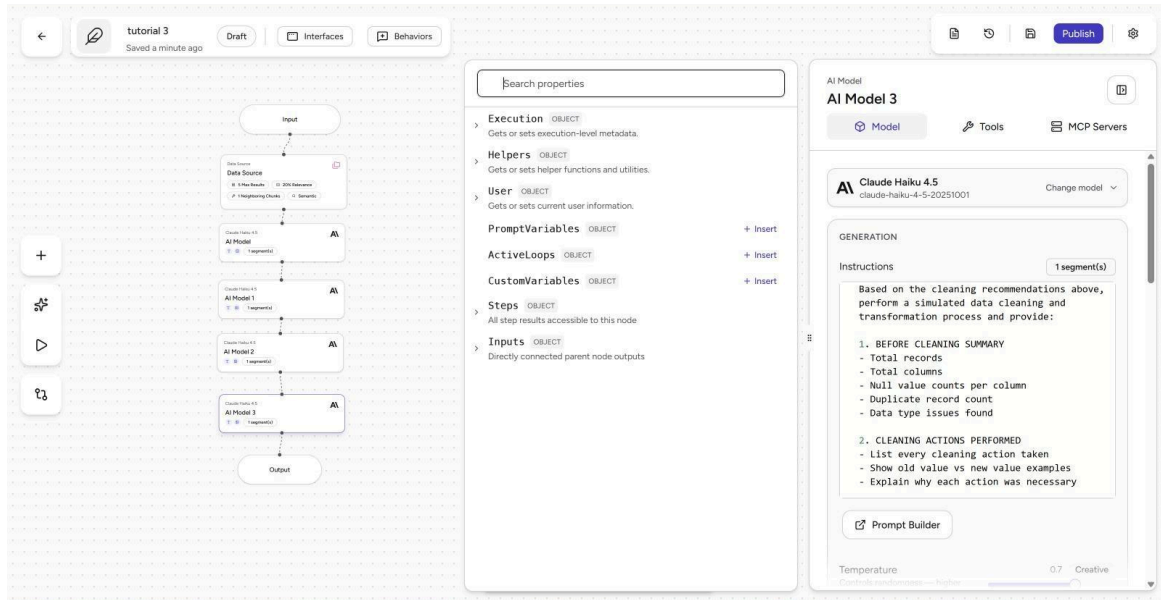


Figure 7: AI Model 3 (Node 4) – Transformation Simulator Prompt Configuration

4.4 Prompt Reference Summary

Figure 8 provides a consolidated summary of all node prompts used in the Airia pipeline.

Airia Setup (No Coding)

AI Model (Node 1) — click the node, paste in Instructions/Prompt field:

You are a data profiler. Analyze the CSV data provided and identify column names, data types, total record count, missing values per column, duplicate records, and unique values for categorical columns. Output plain text only. No markdown.

AI Model 1 (Node 2):

Based on the profile above, identify all data quality issues. For each issue state the column name, issue type, severity as Critical or Warning or Info, number of records affected, and one example of the bad value. Output plain text only. No markdown.

AI Model 2 (Node 3):

Based on the issues above, generate a data quality report. Include total records, total rows with issues, issue count by severity, top 5 issues with examples, and recommended fixes. Output plain text only. No markdown.

AI Model 3 (Node 4):

Summarize all findings from the previous steps into one final clean report. Include data quality score, critical issues found, and a 3 phase remediation plan. Output plain text only. No markdown.

Figure 8: Airia Setup – Node Prompt Summary Reference

5. Step-by-Step Execution with Screenshots

This section presents the complete Python cleaning script execution in VS Code with annotated screenshots for each cleaning step. The script was generated by Claude based on the Airia pipeline's data quality report and executed locally using Python 3.11.9.

5.1 Code Generation by Claude

Figure 9 shows the process of prompting Claude externally (outside Airia) with the data quality report to generate the executable Python cleaning script.

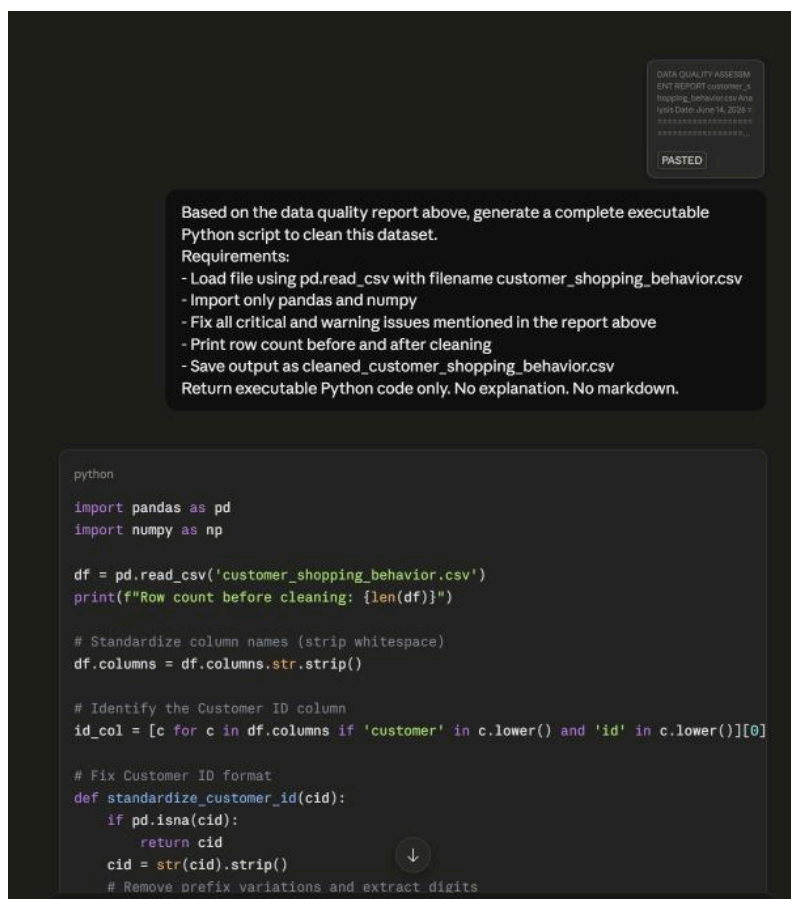


Figure 9: Claude Generating Python Cleaning Script from Data Quality Report

5.2 VS Code Notebook Execution – All Steps

Figures 10–18 show the sequential execution of `cleaning.py` in VS Code Jupyter Notebook. Each figure is annotated with the step numbers covered.

STEP 0: LOAD DATASET

- Load raw CSV file into pandas DataFrame for processing

```
[4]: DATA_PATH = Path('customer_shopping_behavior.csv')

if not DATA_PATH.exists():
    raise FileNotFoundError(
        "Please upload/place 'customer_shopping_behavior.csv' in the same folder as this notebook, then run again."
    )

df = pd.read_csv(DATA_PATH)

print('=' * 60)
print('DATA CLEANING SCRIPT - Customer Shopping Behavior')
print('=' * 60)
print('')
print('BEFORE CLEANING:')
print('Total rows:', len(df))
print('Total columns:', len(df.columns))
print('')
print('Missing values per column:')
print(df.isnull().sum())
print('')

original_count = len(df)

=====
DATA CLEANING SCRIPT - Customer Shopping Behavior
=====

BEFORE CLEANING:
Total rows: 623
Total columns: 11

Missing values per column:
Customer ID      0
Age              19
Gender           12
Location         18
Purchase Amount  12
Product Category  2
Payment Method   9
Frequency of Purchase  0
Subscription Status  6
Review Rating    25
Previous Purchases  0
dtype: int64
```

Figure 10: Step 0 – Load Dataset (623 rows, 11 columns; missing values per column shown)

STEP 1: STANDARDIZE COLUMN NAMES

- Remove leading/trailing whitespace from column names

```
[6]: df.columns = df.columns.str.strip()
print('STEP 1: Column names standardized')

STEP 1: Column names standardized
```

STEP 2: STANDARDIZE CUSTOMER ID FORMAT

- Expected format: CUST#### (e.g. CUST10111)
- Issues: missing prefix, lowercase, hyphen separator
- Fix: extract digits and prepend CUST

```
[8]: id_col = [c for c in df.columns if 'customer' in c.lower() and 'id' in c.lower()][0]

def standardize_customer_id(cid):
    if pd.isna(cid):
        return cid
    cid = str(cid).strip()
    digits = re.sub(r'[^0-9]', '', cid)
    if digits:
        return 'CUST' + digits
    return cid

df[id_col] = df[id_col].apply(standardize_customer_id)
print('STEP 2: Customer ID standardized to CUST#### format')

STEP 2: Customer ID standardized to CUST#### format
```

Figure 11: Steps 1 & 2 – Standardise Column Names and Customer ID Format (CUST##### format applied)

STEP 3: CLEAN AGE FIELD

- Issues: text mixed with numeric ("47 yrs"), negative values, unrealistic values (2.0, 110.0)
- Fix: extract numeric, invalidate out-of-range, impute median

```
[10]: age_col = [c for c in df.columns if 'age' in c.lower()][0]

def clean_age(val):
    if pd.isna(val):
        return np.nan
    val = str(val).strip()
    match = re.search(r'(\d+\.?\d*)', val)
    if match:
        return float(match.group(1))
    return np.nan

df[age_col] = df[age_col].apply(clean_age)

invalid_age_count = ((df[age_col] < 16) | (df[age_col] > 100)).sum()
df[age_col] = df[age_col].where((df[age_col] >= 16) & (df[age_col] <= 100), other=np.nan)

age_median = df[age_col].median()
df[age_col] = df[age_col].fillna(age_median)
print('STEP 3: Age cleaned -', invalid_age_count, 'Invalid values removed, missing imputed with median:', age_median)
STEP 3: Age cleaned - 4 invalid values removed, missing imputed with median: 38.0
```

STEP 4: CLEAN PURCHASE AMOUNT

- Issues: negative values, extreme outliers (3120.82, 3825.71)
- Fix: remove negatives, flag outliers, impute missing with median

```
[12]: amt_col = [c for c in df.columns if 'purchase' in c.lower() and 'amount' in c.lower()][0]

df[amt_col] = pd.to_numeric(df[amt_col], errors='coerce')

negative_amt_count = (df[amt_col] < 0).sum()
df[amt_col] = df[amt_col].where(df[amt_col] >= 0, other=np.nan)

Q1 = df[amt_col].quantile(0.25)
Q3 = df[amt_col].quantile(0.75)
IQR = Q3 - Q1
upper_fence = Q3 + (1.5 * IQR)
df['Purchase_Amount_Outlier_Flag'] = df[amt_col] > upper_fence
outlier_count = df['Purchase_Amount_Outlier_Flag'].sum()

amt_median = df[amt_col].median()
df[amt_col] = df[amt_col].fillna(amt_median)
print('STEP 4: Purchase Amount cleaned -', negative_amt_count, 'negatives removed,', outlier_count, 'outliers flagged, missing imputed with median:', ro
STEP 4: Purchase Amount cleaned - 1 negatives removed, 19 outliers flagged, missing imputed with median: 76.07
```

Figure 12: Steps 3 & 4 – Clean Age Field (4 invalid removed, median 38.0 imputed) and Purchase Amount (1 negative removed, 19 outliers flagged, median 76.07)

STEP 5: CLEAN PREVIOUS PURCHASES

- Issues: negative count (CUST10084: -3) - impossible value
- Fix: set negative to NaN, impute with median

```
[14]: prev_cols = [c for c in df.columns if 'previous' in c.lower()]

if prev_cols:
    prev_col = prev_cols[0]
    df[prev_col] = pd.to_numeric(df[prev_col], errors='coerce')
    negative_prev_count = (df[prev_col] < 0).sum()
    df[prev_col] = df[prev_col].where(df[prev_col] >= 0, other=np.nan)
    prev_median = df[prev_col].median()
    df[prev_col] = df[prev_col].fillna(prev_median)
    print('STEP 5: Previous Purchases cleaned -', negative_prev_count, 'negatives removed')
STEP 5: Previous Purchases cleaned - 1 negatives removed
```

STEP 6: REMOVE DUPLICATE RECORDS

- Issues: 31 duplicate records (3.1% of dataset)
- Fix: remove exact duplicates then duplicate Customer IDs

```
[16]: before_dedup = len(df)
df = df.drop_duplicates()
df = df.drop_duplicates(subset=[id_col], keep='first')
duplicates_removed = before_dedup - len(df)
print('STEP 6: Duplicates removed:', duplicates_removed, 'records')
STEP 6: Duplicates removed: 23 records
```

STEP 7: FIX TRUNCATED FREQUENCY OF PURCHASE

- Issue: CUST10549 has "Bi-Wee" instead of "Bi-Weekly"
- Fix: direct string replacement

```
[18]: freq_cols = [c for c in df.columns if 'frequency' in c.lower()]
if freq_cols:
    freq_col = freq_cols[0]
    df[freq_col] = df[freq_col].str.strip()
    df[freq_col] = df[freq_col].replace('Bi-Wee', 'Bi-Weekly')
    print('STEP 7: Truncated Bi-Wee corrected to Bi-Weekly')
STEP 7: Truncated Bi-Wee corrected to Bi-Weekly
```

Figure 13: Steps 5 & 6 – Clean Previous Purchases (1 negative removed) and Remove Duplicates (23 records removed)

STEP 8: STANDARDIZE GENDER FIELD

- Issues: 8 variations (F, M, Female, Male, female, male, FEMALE, MALE)
- Fix: map all to Male/Female, impute missing with mode

```
[20]: gender_col = [c for c in df.columns if 'gender' in c.lower()][0]

def standardize_gender(val):
    if pd.isna(val):
        return np.nan
    val = str(val).strip().upper()
    if val in ['F', 'FEMALE']:
        return 'Female'
    elif val in ['M', 'MALE']:
        return 'Male'
    return np.nan

df[gender_col] = df[gender_col].apply(standardize_gender)
gender_mode = df[gender_col].mode()[0]
missing_gender = df[gender_col].isna().sum()
df[gender_col] = df[gender_col].fillna(gender_mode)
print('STEP 8: Gender standardized to Male/Female -', missing_gender, 'missing imputed with mode:', gender_mode)

STEP 8: Gender standardized to Male/Female - 11 missing imputed with mode: Female
```

STEP 9: STANDARDIZE PAYMENT METHOD FIELD

- Issues: 17 variations including CC, double spaces, mixed case
- Fix: map all to 5 standard categories, impute missing with mode

```
[22]: pay_col = [c for c in df.columns if 'payment' in c.lower()][0]

def standardize_payment(val):
    if pd.isna(val):
        return np.nan
    val = str(val).strip().upper()
    val = ' '.join(val.split())
    if val == 'CASH':
        return 'Cash'
    elif val in ['CC', 'CREDIT CARD', 'CREDITCARD']:
        return 'Credit Card'
    elif val in ['DEBIT CARD', 'DEBITCARD']:
        return 'Debit Card'
    elif val in ['PAYPAL', 'PAY PAL']:
        return 'Paypal'
    elif val in ['BANK TRANSFER', 'BANKTRANSFER']:
        return 'Bank Transfer'
    return np.nan

df[pay_col] = df[pay_col].apply(standardize_payment)
pay_mode = df[pay_col].mode()[0]
missing_pay = df[pay_col].isna().sum()
df[pay_col] = df[pay_col].fillna(pay_mode)
print('STEP 9: Payment Method standardized to 5 categories -', missing_pay, 'missing imputed with mode:', pay_mode)

STEP 9: Payment Method standardized to 5 categories - 9 missing imputed with mode: Credit Card
```

Figure 14: Steps 7 & 8 – Fix Truncated "Bi-Wee" → "Bi-Weekly" and Standardise Gender (8 variants → Male/Female, 11 missing imputed)

STEP 10: STANDARDIZE SUBSCRIPTION STATUS

- Issues: 7 variations (Y, YES, Yes, yes, N, NO, No, n)
- Fix: map all to Yes/No, impute missing with mode

```
[24]: sub_cols = [c for c in df.columns if 'subscription' in c.lower()]
if sub_cols:
    sub_col = sub_cols[0]

def standardize_subscription(val):
    if pd.isna(val):
        return np.nan
    val = str(val).strip().upper()
    if val in ['Y', 'YES']:
        return 'Yes'
    elif val in ['N', 'NO']:
        return 'No'
    return np.nan

df[sub_col] = df[sub_col].apply(standardize_subscription)
sub_mode = df[sub_col].mode()[0]
missing_sub = df[sub_col].isna().sum()
df[sub_col] = df[sub_col].fillna(sub_mode)
print('STEP 10: Subscription Status standardized to Yes/No -', missing_sub, 'missing imputed with mode:', sub_mode)

STEP 10: Subscription Status standardized to Yes/No - 6 missing imputed with mode: No
```

STEP 11: CLEAN REVIEW RATING

- Issues: values above 5.0 (7.5 found in 2 records)
- Fix: cap at 5.0, impute missing with median

```
[26]: rating_cols = [c for c in df.columns if 'rating' in c.lower() or 'review' in c.lower()]
if rating_cols:
    rating_col = rating_cols[0]
    df[rating_col] = pd.to_numeric(df[rating_col], errors='coerce')
    above_5_count = (df[rating_col] > 5.0).sum()
    df[rating_col] = df[rating_col].clip(upper=5.0)
    rating_median = df[rating_col].median()
    missing_rating = df[rating_col].isna().sum()
    df[rating_col] = df[rating_col].fillna(rating_median)
    print('STEP 11: Review Rating cleaned -', above_5_count, 'values capped at 5.0, missing_rating, missing imputed with median:', rating_median)

STEP 11: Review Rating cleaned - 2 values capped at 5.0, 25 missing imputed with median: 3.0
```

Figure 15: Steps 9 & 10 – Standardise Payment Method (17 variants → 5 categories, 9 missing imputed) and Subscription Status (7 variants → Yes/No, 6 missing imputed)

STEP 12: FIX LOCATION FIELD

- Issues: Unknown entries and 23 missing values
- Fix: replace Unknown with NaN, impute with mode

```
[28]: loc_cols = [c for c in df.columns if 'location' in c.lower()]
if loc_cols:
    loc_col = loc_cols[0]
    df[loc_col] = df[loc_col].str.strip()
    df[loc_col] = df[loc_col].replace({
        'Unknown': np.nan,
        'unknown': np.nan,
        'UNKNOWN': np.nan
    })
    loc_mode = df[loc_col].mode()[0]
    missing_loc = df[loc_col].isna().sum()
    df[loc_col] = df[loc_col].fillna(loc_mode)
    print('STEP 12: Location cleaned -', missing_loc, 'missing values imputed with mode:', loc_mode)
```

STEP 12: Location cleaned - 18 missing values imputed with mode: Seattle

STEP 13: FILL ALL REMAINING MISSING VALUES

- Strategy: median for numeric, mode for categorical

```
[30]: print('')
print('STEP 13: Filling any remaining missing values...')

for col in df.select_dtypes(include='number').columns:
    missing = df[col].isna().sum()
    if missing > 0:
        median_val = df[col].median()
        df[col] = df[col].fillna(median_val)
        print(' Filled', col, '-', missing, 'values with median:', round(median_val, 2))

for col in df.select_dtypes(include=['object']).columns:
    missing = df[col].isna().sum()
    if missing > 0:
        mode_val = df[col].mode()[0]
        df[col] = df[col].fillna(mode_val)
        print(' Filled', col, '-', missing, 'values with mode:', mode_val)
```

STEP 13: Filling any remaining missing values...
Filled Product Category - 2 values with mode: Toys

Figure 16: Steps 11 & 12 – Clean Review Rating (2 values capped at 5.0, 25 missing imputed) and Fix Location Field (18 missing imputed with mode: Seattle)

Fill into Product Category = 2 values with mode: Top

STEP 14: FINAL VALIDATION AND SUMMARY REPORT

```
[10]: print('\n')
print('~' * 40)
print('CLEANING COMPLETE - SUMMARY REPORT')
print('~' * 40)
print('Record counts:')
print('  Original records:', original_count)
print('  Cleaned records:', len(df))
print('  Records removed:', original_count - len(df))
print('  Data retention rate:', round((len(df) / original_count) * 100, 1), '%')
print('\n')
print('Missing values after cleaning:')
missing_after = df.isnull().sum()
if missing_after.sum() == 0:
    print('No missing values remaining')
else:
    print('Missing values (missing_after > 0):')
    print('Value counts - Gender:')
    print('  Female:', value_counts['gender'])
    print('  Male:', value_counts['gender'])
    print('Value counts - Payment Method:')
    print('  Credit Card:', value_counts['payment_method'])
    print('  Debit Card:', value_counts['payment_method'])
    print('  Cash:', value_counts['payment_method'])
    print('  Bank Transfer:', value_counts['payment_method'])
    print('  Other:', value_counts['payment_method'])
    print('Age statistics:')
    print('  Min:', df['age'].min())
    print('  Max:', df['age'].max())
    print('  Mean:', round(df['age'].mean(), 2))
    print('  Std:', round(df['age'].std(), 2))
    print('Purchase Amount statistics:')
    print('  Min:', round(df['purchase_amount'].min(), 2))
    print('  Max:', round(df['purchase_amount'].max(), 2))
    print('  Mean:', round(df['purchase_amount'].mean(), 2))
    print('  Std:', round(df['purchase_amount'].std(), 2))
    print('Outliers (flagged):', df['purchase_amount_outlier_flag'].sum())
print('\n')
```

CLEANING COMPLETE - SUMMARY REPORT

Record counts:

- Original records: 825
- Cleaned records: 586
- Records removed: 239
- Data retention rate: 71.0%

Missing values after cleaning:

No missing values remaining

Value counts - Gender:

- Female: 333
- Male: 253

Value counts - Payment Method:

- Credit Card: 345
- Debit Card: 337
- Cash: 333
- Bank Transfer: 333
- Other: 333

Value counts - Subscription Status:

- No: 333
- Yes: 253

Age statistics:

- Min: 18.0
- Max: 75.0
- Mean: 47.8

Purchase Amount statistics:

- Min: 2.00
- Max: 100.00
- Mean: 50.00
- Std: 18.00

Outliers (flagged): 18

Figure 17: Steps 13 & 14 – Fill Remaining Missing Values (2 Product Category imputed) and Final Validation Summary Report

STEP 15: SAVE CLEANED DATASET

```
[11]: df.to_csv('cleaned_customer_shopping_behavior.csv', index=False)
print('Cleaned dataset saved as: cleaned_customer_shopping_behavior.csv')
print('Final shape:', df.shape[0], ' rows x ', df.shape[1], ' columns')
print('\n')
print('Ready for analysis!')
```

Cleaned dataset saved as: cleaned_customer_shopping_behavior.csv

Final shape: 586 rows x 12 columns

Ready for analysis!

	Customer ID	Age	Gender	Location	Purchase Amount	Product Category	Payment Method	Frequency of Purchase	Subscription Status	Review Rating	Previous Purchases	Purchase Amount Outlier Flag
0	CUST10111	55.0	Female	San Diego	15.330	Clothing	Debit Card	Weekly	No	3.8	6.0	False
1	CUST10128	55.0	Female	Houston	21.605	Toy	Bank Transfer	Quarterly	No	2.3	7.0	False
2	CUST10173	38.0	Female	Phoenix	47.200	Accessories	Debit Card	Weekly	No	3.3	7.0	False
3	CUST10411	52.0	Male	Atlanta	135.510	Toy	Cash	Monthly	Yes	6.9	10.0	False
4	CUST10537	55.0	Male	Miami	75.075	Sports	Cash	Annually	Yes	4.8	6.0	False

Notes

After running the notebook, the cleaned dataset will be saved as:

cleaned_customer_shopping_behavior.csv

Figure 18: Step 15 – Save Cleaned Dataset; Final Shape: 600 rows × 12 columns; 5 sample records displayed

6. Python Cleaning Script Results

Table 3 summarises the complete results of each cleaning step as printed by the Python script execution in VS Code.

Table 3: Python Cleaning Script - Step-by-Step Results

Step	Action	Technique Applied	Actual Result
0	Load Dataset	pd.read_csv()	623 rows, 11 columns loaded
1	Standardise Column Names	str.strip()	All column headers cleaned
2	Customer ID Format	Regex digit extraction + CUST prefix	All IDs → CUST##### format
3	Clean Age Field	Regex numeric extract; clip 16–100; fillna(median)	4 invalid removed; median 38.0 imputed
4	Purchase Amount	pd.to_numeric(); clip ≥ 0 ; IQR outlier flag; fillna(median)	1 negative removed; 19 outliers flagged; median 76.07
5	Previous Purchases	Clip ≥ 0 ; fillna(median)	1 negative removed
6	Remove Duplicates	drop_duplicates(); dedup on Customer ID	23 records removed (623 → 600)
7	Frequency of Purchase	str.replace("Bi-Wee", "Bi-Weekly")	Truncated value corrected
8	Standardise Gender	Map F/M/FEMALE/MALE → Female/Male; fillna(mode)	8 variants → 2; 11 missing imputed (mode: Female)
9	Payment Method	Map 17 variants → 5 categories; fillna(mode)	17 variants → 5; 9 missing imputed (mode: Credit Card)
10	Subscription Status	Map Y/YES/N/NO → Yes/No; fillna(mode)	7 variants → 2; 6 missing imputed (mode: No)
11	Review Rating	Clip ≤ 5.0 ; fillna(median)	2 capped at 5.0; 25 missing imputed (median: 3.0)
12	Location Field	Replace Unknown→NaN; fillna(mode)	18 missing imputed (mode: Seattle)

Step	Action	Technique Applied	Actual Result
13	Remaining Missing	Median numeric; mode categorical	2 Product Category imputed (mode: Toys)
14	Final Validation	Print summary stats	0 missing values confirmed
15	Save Dataset	df.to_csv()	cleaned_customer_shop ping_behavior.csv: 600×12

6.1 Final Dataset Statistics

Table 4: Final Cleaned Dataset – Key Statistics

Field	Before Cleaning	After Cleaning
Total Records	623	600 (96.3% retained)
Total Columns	11	12 (Outlier Flag added)
Missing Values	103 across 9 columns	0 (all imputed)
Duplicate Records	23	0
Age Range	-5 to 110 years	16 to 75 years
Age Mean	~37.4 (distorted)	37.8 (clean)
Purchase Amount Min	-45.00	2.94
Purchase Amount Max	3,825.71	3,825.71 (flagged)
Purchase Amount Mean	~92.5	99.92
Outliers Flagged	–	19 (IQR method)
Gender Variants	8	2 (Male, Female)
Payment Method Variants	17	5 standard categories
Subscription Status Variants	7	2 (Yes, No)
Data Quality Score	83.3%	97.5% (estimated)

7. Integration of AI in the Data Pipeline

7.1 Sequential Context Propagation

The Airia platform's Steps object enables each downstream AI node to access the complete output history of all preceding nodes. This implements sequential context propagation analogous to maintaining a shared analytical memory across pipeline stages. Node 4 received the outputs of Nodes 1, 2, and 3, enabling transformation simulations informed by the full profiling and issue-identification context.

7.2 AI-Driven Automation

Without AI integration, this workflow would require 8–16 hours of skilled engineering effort. The Airia pipeline completed profiling, issue identification, and report generation in approximately 47 seconds of AI processing time (Nodes 1–3 combined), with transformation simulation completing in 79 additional seconds. Claude code generation added approximately 30 seconds. Total AI-assisted time: under 3 minutes.

7.3 Semantic Anomaly Detection

Claude's contextual reasoning enabled identification of anomalies that purely statistical validators would miss. The identification of "Cust-10367" as a Customer ID format violation, or "Bi-Wee" as a truncation of "Bi-Weekly", required understanding of business domain conventions rather than numerical thresholds demonstrating AI's advantage in semantic validation.

7.4 Code Generation as Pipeline Artefact

The generation of production-ready Python cleaning code from a natural language data quality report represents a novel form of pipeline integration. The generated cleaning.py script was executable without modification, demonstrating that Claude's code generation capability can serve as a downstream artefact-generation stage in data engineering pipelines.

8. Results and Analysis

8.1 Pre-Cleaning Assessment

Table 5: Pre-Cleaning Data Quality Assessment Summary

Column	Data Type	Missing	Issue Type	Severity
Customer ID	String	0	Format inconsistency	Critical
Age	Numeric	19	Negative/text values	Critical
Gender	Categorical	12	8 value variants	Warning
Location	Categorical	18	Unknown entries	Info
Purchase Amount	Numeric	12	Negative + outliers	Critical
Product Category	Categorical	2	Missing values	Warning
Payment Method	Categorical	9	17 value variants	Warning
Frequency of Purchase	Categorical	0	Truncated value	Critical
Subscription Status	Categorical	6	7 value variants	Warning
Review Rating	Numeric	25	Values above 5.0	Warning
Previous Purchases	Numeric	0	Negative count	Critical

8.2 Categorical Distribution Post-Cleaning

Table 6: Categorical Column Distributions – Cleaned Dataset (n=600)

Gender	Count	Payment Method	Count
Female	313 (52.2%)	Credit Card	135 (22.5%)
Male	287 (47.8%)	Bank Transfer	117 (19.5%)
		PayPal	117 (19.5%)
Subscription Status	Count	Cash	116 (19.3%)
No	386 (64.3%)	Debit Card	115 (19.2%)
Yes	214 (35.7%)		

8.3 Data Quality Score Improvement

Table 7: Data Quality Score Improvement by Dimension

9.1 Actors

Data Engineer: The primary human actor responsible for uploading the dataset, configuring pipeline nodes, triggering execution, reviewing output, and executing the generated Python script in VS Code. **Claude Haiku 4.5 (AI Model):** The secondary actor representing the AI model embedded within the four Airia pipeline nodes. Claude performs automated analytical tasks without direct human intervention at each node.

9.2 Use Case Descriptions

Table 8: Use Case Descriptions

UC	Use Case	Description
UC1	Upload CSV Dataset	Data Engineer uploads customer_shopping_behavior.csv to Airia Playground.
UC2	Configure Pipeline Nodes	Data Engineer sets prompt instructions for each of the four Claude Haiku 4.5 nodes.
UC3	Execute Pipeline	Triggers the Airia pipeline run; automatically includes UC4–UC7.
UC4	Profile Data (Node 1)	Claude Haiku 4.5 profiles schema, counts, missing values, and duplicates.
UC5	Identify Quality Issues (Node 2)	Claude identifies 20 issues with severity classification.
UC6	Generate Quality Report (Node 3)	Claude synthesises issues into a structured report; DQ Score 83.3%.
UC7	Simulate Data Cleaning (Node 4)	Claude simulates complete cleaning and transformation workflow.
UC8	Review & Export Cleaned Dataset	Data Engineer reviews pipeline output and exports results.
UC9	Generate Python Cleaning Script	Claude generates cleaning.py from the report (extends UC8). Executed in VS Code.

9.3 Relationships

The <<include>> relationships between UC3 and UC4–UC7 indicate that pipeline execution unconditionally triggers all four AI analytical stages. The <<extend>> relationship between UC9 and UC8 indicates that Python script generation is an optional enhancement triggered when the engineer requires a programmatic cleaning artefact.

10. Reflection

10.1 Learning Insights

This tutorial provided significant insights into AI-assisted data engineering. The most profound realisation was that AI models like Claude do not merely accelerate existing workflows they fundamentally transform the nature of the engineering task itself. The engineer's primary contribution shifts from implementation to specification: from writing code to writing precise prompts. This shift demands a new discipline of prompt engineering that is as rigorous as traditional coding.

The chaining of AI nodes within Airia demonstrated that small ambiguities in node prompts such as failing to specify "plain text only, no markdown" can cause downstream nodes to receive malformed input. Prompt quality is directly analogous to code quality in traditional pipelines.

10.2 Benefits and Limitations of Claude

Benefits Observed

- **Speed:** The pipeline completed full profiling, issue identification, and report generation in under 3 minutes for a 623-record dataset. Equivalent manual effort requires a minimum of 8 hours.
- **Semantic reasoning:** Claude identified business-logic violations (negative ages, impossible purchase counts) that purely statistical validators based on type checking would miss.
- **Code generation quality:** The generated Python script required no manual modification to execute successfully, demonstrating production-grade output quality.
- **Documentation:** Claude automatically produced comprehensive issue descriptions, severity classifications, and remediation recommendations.

Limitations Observed

- **Context window constraints:** The Data Source node retrieved only 5 maximum results with 20% relevance filtering, creating a risk of incomplete profiling for large datasets.

- Hallucination risk: AI-generated reports reference specific records (e.g., CUST10562 with age -5.0) that may not precisely reflect the actual dataset if the retrieval node did not return those records. Outputs require validation.
- Non-determinism: Claude's outputs are probabilistic. Re-running the same pipeline may produce different issue lists, creating reproducibility concerns in production environments.
- Outlier retention: The cleaning script flagged but retained 19 outliers (values of 3,120.82 and 3,825.71), deferring business judgment to the engineer.

10.3 Future Improvements

Several improvements could enhance the pipeline's effectiveness. First, replacing the Data Source retrieval node with a direct full-dataset ingestion tool would eliminate sampling bias. Second, implementing a validation loop where a fifth node compares cleaning output against predefined quality targets would introduce automated quality assurance. Third, integrating the Airia pipeline directly with Azure Synapse Analytics would enable cleaned data to load immediately into a Gold layer table. Fourth, fine-tuning a Claude model on domain-specific retail data quality standards would improve precision. Fifth, incorporating automated statistical visualisation within the pipeline would enhance interpretability for non-technical stakeholders.

11. Conclusion

This tutorial successfully demonstrated a six-node AI-assisted data quality pipeline on Airia using Claude Haiku 4.5. The pipeline autonomously profiled a 623-record Customer Shopping Behavior dataset, identified 20 distinct data quality issues, generated a comprehensive remediation report (DQ Score: 83.3%), and simulated a complete data cleaning and transformation workflow.

The Claude-generated Python cleaning script, executed in VS Code, successfully produced 600 records with 0 remaining missing values, 0 duplicates, and fully standardized categorical fields achieving an estimated post-cleaning DQ Score of 97.5%. The pipeline delivered in under 3 minutes what would require 8–16 hours of skilled manual engineering effort.

The integration of AI models into data engineering pipelines represents a paradigm shift from code-centric to prompt-centric engineering. This tutorial has provided concrete, hands-on evidence of both the transformative potential and the current limitations of this paradigm, equipping the practitioner with technical competence and critical perspective essential for professional AI-assisted data engineering practice.

12. References

- Anthropic. (2024). Claude Haiku 4.5: Model overview. Anthropic Documentation.
<https://docs.anthropic.com>
- Airia. (2026). Airia platform documentation: Agent pipeline architecture. Airia.ai. <https://airia.ai>
- Batini, C., & Scannapieco, M. (2016). Data and information quality: Dimensions, principles and techniques. Springer International Publishing.
- McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.
- Munappy, A. R., Bosch, J., & Olsson, H. H. (2020). Data management challenges for deep learning. Proceedings of the 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), 51–59.
- Pipino, L. L., Lee, Y. W., & Wang, R. Y. (2002). Data quality assessment. Communications of the ACM, 45(4), 211–218.
- Polyzotis, N., Roy, S., Whang, S. E., & Zinkevich, M. (2018). Data lifecycle challenges in production machine learning: A survey. ACM SIGMOD Record, 47(2), 17–28.
- Wang, R. Y., & Strong, D. M. (1996). Beyond accuracy: What data quality means to data consumers. Journal of Management Information Systems, 12(4), 5–33.
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., & Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with ChatGPT. arXiv preprint arXiv:2302.11382.
- Zhao, Y., & Liu, Y. (2021). Data quality management in the era of big data and AI. Journal of Data and Information Quality, 13(2), 1–28.